

Vector Space Model

Introduction to Information Retrieval

Dwaipayan Roy

Department of Computational and Data Sciences



Think-Pair-Share: Quantifying Importance

Consider a corpus of 1000 documents ($N=1000$).

- Term **computer** appears in 800 documents.
- Term **algorithm** appears in 100 documents.
- Term **quantum** appears in 5 documents.

Think-Pair-Share: Quantifying Importance

Consider a corpus of 1000 documents ($N=1000$).

- Term **computer** appears in 800 documents.
- Term **algorithm** appears in 100 documents.
- Term **quantum** appears in 5 documents.

Question

For a query "**quantum computer algorithm**", which term do you think is the *most* important for identifying relevant documents, and which is the *least* important?

- Ⓐ **computer** is most important; **quantum** is least.
- Ⓑ **quantum** is most important; **computer** is least.
- Ⓒ **algorithm** is most important; **computer** is least.
- Ⓓ They are all equally important.

Best-match retrieval models

beyond exact-match retrieval models

- Best-match models predict the degree to which a document is relevant to a query.
- $\text{relevance}(d,Q) \sim \text{similarity}(d,Q)$.
- How to compute the similarity between Q and d ? $\rightarrow$ topic of interest.
- For each d in C :
 - ▶ assign a score to document $d = \text{score}(d,Q)$

Introduction to Best-Match Retrieval Models

- Predicts the degree of relevance of a document to a query
- ... expressed as $\text{similarity}(Q,d)$

Vector Space

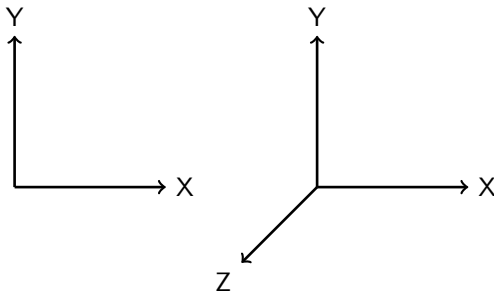
Vector space

- A set of **vectors** closed under addition and multiplications.

Vector Space

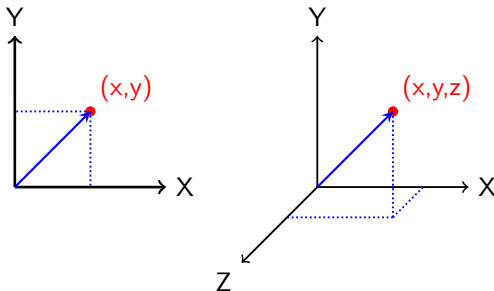
Vector space

- A set of **vectors** closed under addition and multiplications.
- Can also be defined as a set of linearly independent **basis vectors**.
 - ▶ correspond to the dimension of the vector space.



Vector

- **Vector**: a point in a vector space with a **length** (from the origin to the point) and a **direction**.



Incidence vectors

Binary Representation of Text

- **Term-incidence vector:** *for each term t , a vector indicating the documents t appears in.*
- **Document-incidence vector:** *for each document d , a vector indicating the terms occurring in d .*

	Inception	The Dark Knight	Interstellar	Dunkirk	The Prestige	Memento
duality	0	1	0	0	1	0
identity	0	1	0	0	1	1
memory	1	0	0	0	0	1
rivalry	0	1	0	0	1	0
sacrifice	0	0	1	1	1	0
survival	0	0	1	1	0	0
time	1	0	1	1	0	1

tiv(sacrifice)

001110: **sacrifice** appears in docs 3, 4, 5

div(Inception)

0010001: **Inception** has the terms: 3, 7

Term-document incidence matrix

- Dimension of the vector space $\rightarrow$ number of unique terms in the vocabulary.

Incidence vectors

Binary Representation of Text

- **Term-incidence vector:** *for each term t , a vector indicating the documents t appears in.*
- **Document-incidence vector:** *for each document d , a vector indicating the terms occurring in d .*

	Inception	The Dark Knight	Interstellar	Dunkirk	The Prestige	Memento
duality	0	1	0	0	1	0
identity	0	1	0	0	1	1
memory	1	0	0	0	0	1
rivalry	0	1	0	0	1	0
sacrifice	0	0	1	1	1	0
survival	0	0	1	1	0	0
time	1	0	1	1	0	1

tiv(sacrifice)

001110: sacrifice appears in docs 3, 4, 5

div(Inception)

0010001: Inception has the terms: 3, 7

Term-document incidence matrix

- Dimension of the vector space $\rightarrow$ number of unique terms in the vocabulary.
- 1 = document contains the word (one or more times).
- 0 = document doesn't contain the word.

Incidence vectors

Binary Representation of Text

- **Term-incidence vector:** *for each term t , a vector indicating the documents t appears in.*
- **Document-incidence vector:** *for each document d , a vector indicating the terms occurring in d .*

	Inception	The Dark Knight	Interstellar	Dunkirk	The Prestige	Memento
duality	0	1	0	0	1	0
identity	0	1	0	0	1	1
memory	1	0	0	0	0	1
rivalry	0	1	0	0	1	0
sacrifice	0	0	1	1	1	0
survival	0	0	1	1	0	0
time	1	0	1	1	0	1

tiv(sacrifice)

001110: **sacrifice** appears in docs 3, 4, 5

div(Inception)

0010001: **Inception** has the terms: 3, 7

Term-document incidence matrix

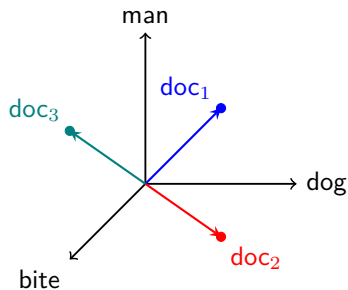
- Dimension of the vector space $\rightarrow$ number of unique terms in the vocabulary.
- 1 = document contains the word (one or more times).
- 0 = document doesn't contain the word.
- No information on word frequency, location of occurrence etc.

Vector Space

Binary Weights

Let, $V = \{\text{bite, dog, man}\}$

- $\text{doc}_1 = \text{"dog bite man"} \Rightarrow (1, 1, 1)$
- $\text{doc}_2 = \text{"dog bite"} \Rightarrow (0, 1, 1)$
- $\text{doc}_3 = \text{"man bite"} \Rightarrow (1, 0, 1)$

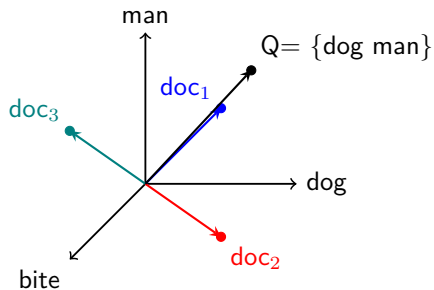


Vector Space

Binary Weights

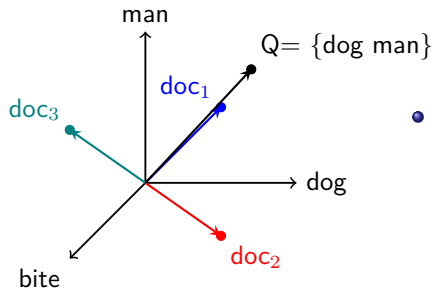
Let, $V = \{\text{bite, dog, man}\}$

- $\text{doc}_1 = \text{"dog bite man"} \Rightarrow (1, 1, 1)$
- $\text{doc}_2 = \text{"dog bite"} \Rightarrow (0, 1, 1)$
- $\text{doc}_3 = \text{"man bite"} \Rightarrow (1, 0, 1)$



Vector Space Model (VSM)

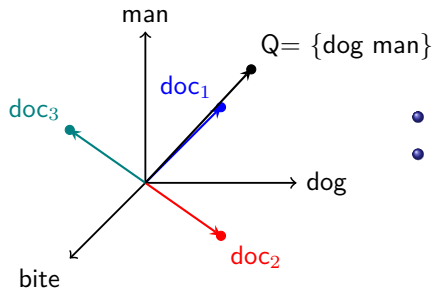
Retrieval using VSM



- Represent the query and document in the same vector space.

Vector Space Model (VSM)

Retrieval using VSM



- Represent the query and document in the same vector space.
- Compute the **similarity** between vectors.
 - ▶ will produce a ranked list of documents.

Computing Similarity in Vector Space Model

1. Inner Product

- For a query Q and document d , retrieval $\text{score}(Q, d)$: $\text{score}(Q, d) = \sum_{q \in Q} w(q, d)$

Computing Similarity in Vector Space Model

1. Inner Product

- For a query Q and document d , retrieval $\text{score}(Q, d)$: $\text{score}(Q, d) = \sum_{q \in Q} w(q, d)$

Similarity in vector space

- Vector-space similarity between query vector and document vector:

$$\text{score}(Q, d) = \text{vecsims}(\mathbf{Q}, \mathbf{d})$$

- Define $\text{vecsims}(\mathbf{X}, \mathbf{Y})$ where: $\mathbf{X} = [x_1 x_2 \dots x_{dim}]$ $\mathbf{Y} = [y_1 y_2 \dots y_{dim}]$

Computing Similarity in Vector Space Model

1. Inner Product

- For a query Q and document d , retrieval $\text{score}(Q, d)$: $\text{score}(Q, d) = \sum_{q \in Q} w(q, d)$

Similarity in vector space

- Vector-space similarity between **query vector** and **document vector**:

$$\text{score}(Q, d) = \text{vecsims}(\mathbf{Q}, \mathbf{d})$$

- Define $\text{vecsims}(\mathbf{X}, \mathbf{Y})$ where: $\mathbf{X} = [x_1 x_2 \dots x_{dim}]$ $\mathbf{Y} = [y_1 y_2 \dots y_{dim}]$
 - as the **inner-product** (also called **dot-product**):

$$\text{inner-product}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^{dim} x_i \times y_i$$

- signifies how much the vectors are *aligned*.

$$\mathbf{X} \cdot \mathbf{Y} = \mathbf{X}^T \mathbf{Y} = \sum_{i=1}^{dim} x_i \times y_i$$

Computing Similarity in Vector Space Model

1. Inner Product

	x_i	y_i	$x_i \times y_i$
a	1	1	1
aardvark	0	1	0
abacus	1	1	1
abba	1	0	0
able	0	1	0
::	::	::	::
zoom	0	0	0
inner product $\Rightarrow$			2

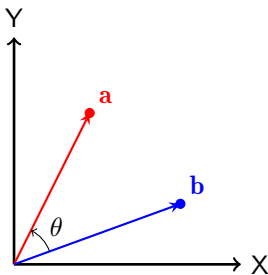
$$\text{inner-product}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^{\dim} x_i \times y_i$$

- Multiply corresponding components and then sum those products.
- Both vectors in same direction $\Rightarrow$ highest dot product.
- $\text{inner-product}(\mathbf{X}, \mathbf{Y}) = 0 \Rightarrow \mathbf{X}, \mathbf{Y}$ orthogonal to each other
 - ▶ no words in common.
- Using a binary representation, the inner product corresponds to the number of terms appearing (at least once) in both text.

Computing Similarity in Vector Space Model

1. Inner Product: Geometrically

- $\mathbf{a} = [a_1 \ a_2]$
- $\mathbf{b} = [b_1 \ b_2]$



$$\mathbf{a}^T \mathbf{b} = a_1 b_1 + a_2 b_2 = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta)$$

Computing Similarity in Vector Space Model

1. Inner Product: Relation to Set Intersection in Binary Text Representation

- set = binary vector: $a_i = 1$ if word i occurs in $\mathbf{a}$; otherwise $a_i = 0$
- $\mathbf{a}^T \mathbf{b} = a_1 b_1 + \dots + a_{dim} b_{dim} = \#$ words in both $\mathbf{a}$ and $\mathbf{b}$

Computing Similarity in Vector Space Model

2. Euclidean Distance

- For a query Q and document d , retrieval $\text{score}(Q, d)$: $\text{score}(Q, d) = \sum_{q \in Q} w(q, d)$

Similarity in vector space

- Vector-space similarity between query vector and document vector:

$$\text{score}(Q, d) = \text{vecsims}(\mathbf{Q}, \mathbf{d})$$

- Define $\text{vecsims}(\mathbf{X}, \mathbf{Y})$ where: $\mathbf{X} = [x_1 x_2 \dots x_{dim}]$ $\mathbf{Y} = [y_1 y_2 \dots y_{dim}]$
 - as the **Euclidean distance**: distance between the endpoints of the vectors.

$$\text{Euclidean-distance}(\mathbf{X}, \mathbf{Y}) = \sqrt{\sum_{i=1}^{dim} (x_i - y_i)^2}$$

Computing Similarity in Vector Space Model

2. Euclidean Distance

- For a query Q and document d , retrieval $\text{score}(Q, d)$: $\text{score}(Q, d) = \sum_{q \in Q} w(q, d)$

Similarity in vector space

- Vector-space similarity between query vector and document vector:

$$\text{score}(Q, d) = \text{vecsims}(\mathbf{Q}, \mathbf{d})$$

- Define $\text{vecsims}(\mathbf{X}, \mathbf{Y})$ where: $\mathbf{X} = [x_1 x_2 \dots x_{dim}]$ $\mathbf{Y} = [y_1 y_2 \dots y_{dim}]$
 - as the **Euclidean distance**: distance between the endpoints of the vectors.

$$\text{Euclidean-distance}(\mathbf{X}, \mathbf{Y}) = \sqrt{\sum_{i=1}^{dim} (x_i - y_i)^2}$$

- signifies how *near* are the vectors in the vector space.

1. Inner Product: The Length Bias Problem

The Relevance Dilemma

Consider query $Q = \{q_1, q_2, q_3\}$:

- **Document A**: 20 words containing **3 query terms**.
- **Document B**: 200 words containing **3 query terms**.

Which document is more relevant to the query?

1. Inner Product: The Length Bias Problem

The Relevance Dilemma

Consider query $Q = \{q_1, q_2, q_3\}$:

- **Document A**: 20 words containing **3 query terms**.
- **Document B**: 200 words containing **3 query terms**.

Which document is more relevant to the query?

The Problem

- Longer documents **more likely** to contain query terms by chance.
- Inner product: $Q \cdot d = \sum q_i \cdot d_i$
- **Favors long documents** with same term matches.
- **Does not normalize** for document length.

Mathematical Issue

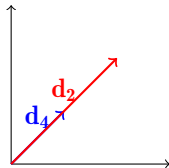
$$\|d_{\text{long}}\| > \|d_{\text{short}}\|$$

as a result
 $\text{score} \propto \|d\|$

2. Euclidean Distance: The Scaling Problem

Fundamental Relationship

- Euclidean distance: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum (x_i - y_i)^2}$
- **Similarity $\propto$ inversely proportional to distance.**
- **Highly sensitive to vector magnitudes.**
- Smaller distance = Higher similarity.
- Larger distance = Lower similarity.

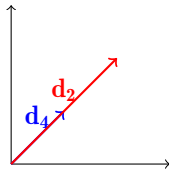


Same direction, different magnitude

2. Euclidean Distance: The Scaling Problem

Fundamental Relationship

- Euclidean distance: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum (x_i - y_i)^2}$
- **Similarity** $\propto$ **inversely proportional to distance.**
- **Highly sensitive to vector magnitudes.**
- Smaller distance = Higher similarity.
- Larger distance = Lower similarity.



Same direction, different magnitude

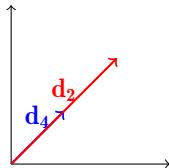
The Scaling Problem

- Consider d_2 which is $2 \times d_4$ (scaled version).
- **Semantically:** d_2 and d_4 have **identical content.**

2. Euclidean Distance: The Scaling Problem

Fundamental Relationship

- Euclidean distance: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum (x_i - y_i)^2}$
- **Similarity** $\propto$ **inversely proportional to distance.**
- **Highly sensitive to vector magnitudes.**
- Smaller distance = Higher similarity.
- Larger distance = Lower similarity.



Same direction, different magnitude

The Scaling Problem

- Consider d_2 which is $2 \times d_4$ (scaled version).
- **Semantically:** d_2 and d_4 have **identical content.**

The Issue

- **Large Euclidean distance** despite **identical semantic meaning.**
- Distance measures **magnitude difference**, not **directional similarity.**

Solution: Cosine Similarity

Angle Over Distance: A Better Similarity Measure

- When angle between two documents **0 degrees** → **maximal similarity**.
- Captures **semantic alignment** rather than size.

Solution: Cosine Similarity

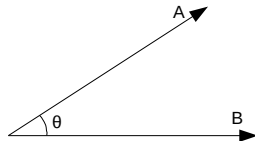
Angle Over Distance: A Better Similarity Measure

- When angle between two documents **0 degrees** → **maximal similarity**.
- Captures **semantic alignment** rather than size.

Core Ranking Principle

Rank documents according to angle with query vector

Smaller angle = Higher relevance



Cosine Similarity Advantage

- $\cos 0^\circ = 1$ (maximum similarity)
- $\cos 90^\circ = 0$ (no similarity)
- $\cos 180^\circ = -1$ (opposite meaning)
- **Scale invariant:** Ignores document length

Solution: Cosine Similarity

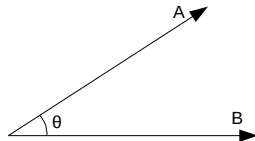
Angle Over Distance: A Better Similarity Measure

- When angle between two documents **0 degrees** → **maximal similarity**.
- Captures **semantic alignment** rather than size.

Core Ranking Principle

Rank documents according to angle with query vector

Smaller angle = Higher relevance



Cosine Similarity Advantage

- $\cos 0^\circ = 1$ (maximum similarity)
- $\cos 90^\circ = 0$ (no similarity)
- $\cos 180^\circ = -1$ (opposite meaning)
- **Scale invariant**: Ignores document length

Practical Implementation

Rank documents in **decreasing order** of $\cos(\theta) = \text{cosine}(Q, d)$

- $\cos(\theta) = 1$: Perfect match
- $\cos(\theta) = 0$: No relationship
- $\cos(\theta) = -1$: Opposite meaning

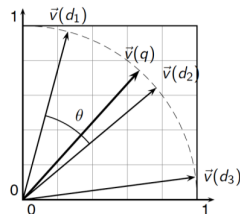
Length Normalization: Implementing Angle-Based Similarity

The Normalization Process

- Normalize vectors by dividing each component by its L_2 **norm**:

$$||\mathbf{x}||_2 = \sqrt{\sum_i x_i^2}$$

- Creates **unit vectors** with magnitude = 1
- Projects all documents onto the **surface of a unit hypersphere**



All vectors normalized to unit length

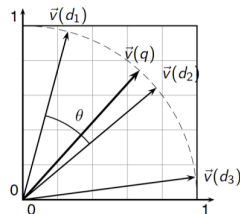
Length Normalization: Implementing Angle-Based Similarity

The Normalization Process

- Normalize vectors by dividing each component by its L_2 **norm**:

$$||\mathbf{x}||_2 = \sqrt{\sum_i x_i^2}$$

- Creates **unit vectors** with magnitude = 1
- Projects all documents onto the **surface of a unit hypersphere**



All vectors normalized to unit length

The Scaling Problem Solved

- $d_2 = 2 \times d_4$ (same content, different lengths)
- **After normalization:**
 - ▶ $\mathbf{d}_2$ and $\mathbf{d}_4$ become **identical vectors**
 - ▶ Same direction = Same semantic meaning
 - ▶ ✓ Length bias eliminated

Cosine Similarity = Normalized Dot Product

$$\text{cos-sim}(Q, d) = \frac{Q \cdot d}{||Q||_2 \cdot ||d||_2} = Q_{\text{norm}} \cdot d_{\text{norm}}$$

Dot product of **length-normalized** vectors =
Cosine of angle between them

Cosine Similarity

Geometric Interpretation

Dot product relationship between two vectors: $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \cdot \|\mathbf{y}\| \cdot \cos \theta$

Cosine Similarity

Geometric Interpretation

Dot product relationship between two vectors: $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \cdot \|\mathbf{y}\| \cdot \cos \theta$

Normalized Formula

Solving for cosine similarity:

$$\cos \theta = \frac{\mathbf{x}}{\|\mathbf{x}\|} \cdot \frac{\mathbf{y}}{\|\mathbf{y}\|}$$

Cosine Similarity

Geometric Interpretation

Dot product relationship between two vectors: $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \cdot \|\mathbf{y}\| \cdot \cos \theta$

Normalized Formula

Solving for cosine similarity:

$$\cos \theta = \frac{\mathbf{x}}{\|\mathbf{x}\|} \cdot \frac{\mathbf{y}}{\|\mathbf{y}\|}$$

Computational Formula

$$\text{cos-sim}(\mathbf{X}, \mathbf{Y}) = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \times \sqrt{\sum_{i=1}^n y_i^2}}$$

Cosine Similarity

Geometric Interpretation

Dot product relationship between two vectors: $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \cdot \|\mathbf{y}\| \cdot \cos \theta$

Normalized Formula

Solving for cosine similarity:

$$\cos \theta = \frac{\mathbf{x}}{\|\mathbf{x}\|} \cdot \frac{\mathbf{y}}{\|\mathbf{y}\|}$$

Computational Formula

$$\text{cos-sim}(\mathbf{X}, \mathbf{Y}) = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \times \sqrt{\sum_{i=1}^n y_i^2}}$$

Key Properties

- **Range:** $[-1, 1]$ or $[0, 1]$ for non-negative vectors.
- **Angle-based:** Measures orientation, not magnitude.
- **Scale invariant:** Unaffected by vector length.
- **Symmetric:** $\text{cos-sim}(A, B) = \text{cos-sim}(B, A)$.

Interpretation

- 1.0: Identical direction
- 0.0: Orthogonal (unrelated)
- -1.0: Opposite direction

Cosine Similarity

Geometric Interpretation

Dot product relationship between two vectors: $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \cdot \|\mathbf{y}\| \cdot \cos \theta$

Normalized

Solving for cosine

cos

Applications in Text Mining

- **Document similarity:** Compare TF-IDF vectors
- **Information retrieval:** Query-document matching
- **Clustering:** Group similar documents
- **Recommendation systems:** User/item similarity

Computational

$$\text{cos-sim}(\mathbf{X}, \mathbf{Y}) = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \times \sqrt{\sum_{i=1}^n y_i^2}}$$

Interpretation

- 1.0: Identical direction
- 0.0: Orthogonal (unrelated)
- -1.0: Opposite direction

vectors.
magnitude.
length.
A).

Vector Space Similarity

example

$$\frac{\sum_{i=1}^{dim} x_i \times y_i}{\sqrt{\sum_{i=1}^{dim} x_i^2} \times \sqrt{\sum_{i=1}^{dim} y_i^2}}$$

$$cosine([1, 0, 1], [1, 1, 0]) =$$

Vector Space Similarity

example

$$\frac{\sum_{i=1}^{dim} x_i \times y_i}{\sqrt{\sum_{i=1}^{dim} x_i^2} \times \sqrt{\sum_{i=1}^{dim} y_i^2}}$$
$$cosine([1, 0, 1], [1, 1, 0]) = \frac{(1 \times 1) + (0 \times 1) + (1 \times 0)}{\sqrt{1^2 + 0^2 + 1^2} \times \sqrt{1^2 + 1^2 + 0^2}} = 0.5$$

Vector Space Similarity

calculation example

- Query: **jill**

doc_1	Jack and Jill went up the hill
doc_2	To fetch a pail of water
doc_3	Jack fell down and broke his crown
doc_4	And Jill came tumbling after
doc_5	Up Jack got, and home did trot
doc_6	As fast as he could caper
doc_7	To old Dame Dob, who patched his nob
doc_8	With vinegar and brown paper

- ? inner product?
- ? cosine similarity?

Cosine Similarity: Query-Document Scoring

Similarity Formula

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|}$$

Vector Computation

$$= \frac{\sum_{i=1}^{|V|} Q_i \cdot d_i}{\sqrt{\sum Q_j^2} \cdot \sqrt{\sum d_j^2}}$$

Term-based Calculation

$$= \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

Cosine Similarity: Query-Document Scoring

Similarity Formula

$$\text{score}(Q, d) = \cos(\theta) = \frac{Q}{\|Q\|} \cdot \frac{d}{\|d\|}$$

Vector Computation

$$= \frac{\sum_{i=1}^{|V|} Q_i \cdot d_i}{\sqrt{\sum Q_j^2} \cdot \sqrt{\sum d_j^2}}$$

Term-based Calculation

$$= \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

Key Insight

- $w(q, Q)$, $w(q, d)$: TF-IDF weights of term q in Query and Document
- Only query terms contribute to the score (sparse computation)

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 1: Binary** (presence / absence)
 - ▶ $\text{weight}(q, d) = 1$ if t present in d ; otherwise 0

$$\text{score}(Q, d) = \sum_q 1_{q \in Q} \cdot 1_{q \in d}$$

- ▶ Document = binary vector = set
- ▶ Similarity $\approx$ word overlap (intersection)

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 1: Binary** (presence / absence)

- ▶ $\text{weight}(q, d) = 1$ if t present in d ; otherwise 0

$$\text{score}(Q, d) = \sum_q 1_{q \in Q} \cdot 1_{q \in d}$$

- ▶ Document = binary vector = set
- ▶ Similarity $\approx$ word overlap (intersection)
- ▶ Problem - **No notion of ranking.**

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 2: Weighted** (keywords tend to repeat in documents)

- ▶ $\text{weight}(t, d) = \text{tf}(t, d)$: # occurrence of t in d

$$\text{score}(Q, d) = \sum_q \text{tf}(q, Q) \cdot \text{tf}(q, d)$$

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 2: Weighted** (keywords tend to repeat in documents)

- ▶ $\text{weight}(t, d) = \text{tf}(t, d)$: # occurrence of t in d

$$\text{score}(Q, d) = \sum_q \text{tf}(q, Q) \cdot \text{tf}(q, d)$$

- ▶ Consider a collection consisting of **tweets** and **Wikipedia articles** of related topics

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 2: Weighted** (keywords tend to repeat in documents)

- ▶ $\text{weight}(t, d) = \text{tf}(t, d)$: # occurrence of t in d

$$\text{score}(Q, d) = \sum_q \text{tf}(q, Q) \cdot \text{tf}(q, d)$$

- ▶ Consider a collection consisting of **tweets** and **Wikipedia articles** of related topics
 - ★ **tweets** (almost) never retrieved

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 2: Weighted** (keywords tend to repeat in documents)

- ▶ $\text{weight}(t, d) = \text{tf}(t, d)$: # occurrence of t in d

$$\text{score}(Q, d) = \sum_q \text{tf}(q, Q) \cdot \text{tf}(q, d)$$

- ▶ Consider a collection consisting of **tweets** and **Wikipedia articles** of related topics
 - ★ **tweets** (almost) never retrieved
- ▶ Problem -
 - ★ **Biased towards long documents;**
 - ★ Have to **normalize by document length.**

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 3: Normalized term frequency** (considers relative tf)

- ▶ $\text{weight}(t, d) = \frac{tf(t, d)}{|d|}$: (length) normalized tf of t in d

$$\text{score}(Q, d) = \sum_t tf(t, Q) \cdot \frac{tf(t, d)}{|d|}$$

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 3:** Normalized term frequency (considers relative tf)

- ▶ $\text{weight}(t, d) = \frac{tf(t, d)}{|d|}$: (length) normalized tf of t in d

$$\text{score}(Q, d) = \sum_t tf(t, Q) \cdot \frac{tf(t, d)}{|d|}$$

- ▶ Why not $\frac{tf(t, Q)}{|Q|}$?

Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 4:** Rare / discriminating term weights:

- ▶ $\text{weight}(t, d) = \frac{tf(t, d)}{|d|} \cdot \log \frac{|N|}{df(t)}$
 - ★ $|N|$ = number of document in the collection
 - ★ $df(t)$ = number of document containing the term t
- ▶ Give more weights to rare terms: $\log \frac{|N|}{df(t)}$

$$\text{score}(Q, d) = \sum_t tf(t, Q) \cdot \frac{tf(t, d)}{|d|} \cdot \log \frac{|N|}{df(t)}$$

Term Weighting: Beyond Raw Frequency

Understanding Similarity Calculation Challenges

The Problem with Raw Term Counts

- Suppose query term q occurs **twice** in Doc_A and **once** in Doc_B
- Is Doc_A **twice as relevant** as Doc_B ?

The Extreme Case

- Suppose query term q occurs **20 times** in Doc_A and **once** in Doc_B
- Is Doc_A **twenty times more relevant** than Doc_B ?

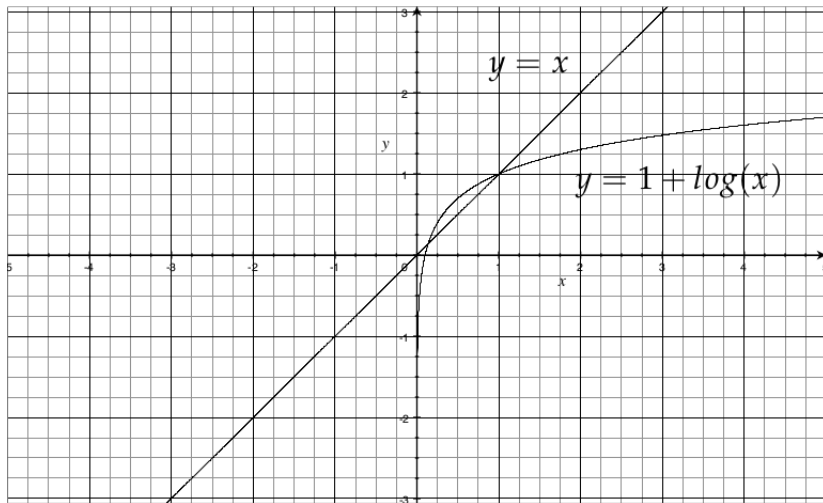
Issues with Raw TF

- **Non-linear relevance:** 20x frequency $\neq$ 20x relevance
- **Document length bias:** Longer docs have higher counts
- **Saturation effect:** Diminishing returns after certain frequency

Better Approaches

- **Logarithmic TF:** $1 + \log(tf)$
- **Double log TF:** $1 + \log(\log(tf))$
- **Saturated TF:** $\frac{tf}{k+tf}$

Sub-linear TF scaling



Term Weighting

similarity calculation

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

- **Approach 5:** Sub-linear term frequency scaling

- ▶ $\text{weight}(t, d) = \left(1 + \log\left(\frac{tf(t, d)}{|d|}\right)\right) \cdot \log \frac{|N|}{df(t)}$

$$\text{score}(Q, d) = \sum_t tf(t, Q) \cdot \left(1 + \log\left(\frac{tf(t, d)}{|d|}\right)\right) \cdot \log \frac{|N|}{df(t)}$$

Cosine similarity between Query and Document

$$\text{score}(Q, d) = \cos(\theta) = \frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \frac{\mathbf{d}}{\|\mathbf{d}\|} = \sum_{i=1}^{|V|} \frac{Q_i \times d_i}{\sqrt{Q_j^2} \times \sqrt{d_j^2}} = \sum_{q \in Q} w(q, Q) \cdot w(q, d)$$

- $w(q, Q), w(q, d) \Rightarrow$ weights of term q in Query and Document respectively.

Cosine similarity between Query and Document

Algorithm 2: Vector Space Model Scoring

Input: Query Q

Output: Top K ranked documents

initialize $\text{scores}[N] \leftarrow 0$

initialize $\text{length}[N]$

initialize $\text{selectedDocs}[N] \leftarrow \text{empty set}$

foreach $q \in Q$ **do**

 calculate $w(q, Q)$

$pl \leftarrow \text{getPostingList}(q)$

$\text{selectedDocs.union}(pl)$

foreach $d \in pl$ **do**

$\text{scores}[d] \leftarrow \text{scores}[d] + w(q, d) \times w(q, Q)$

end

end

foreach $d \in \text{selectedDocs}$ **do**

$\text{scores}[d] \leftarrow \text{scores}[d] / \text{length}[d]$

end

sort $\text{scores}[]$

return top K documents with highest $\text{scores}[]$

$\text{score}(Q, d)$

• $w(q, Q), w(q, d) \Rightarrow$

$Q) \cdot w(q, d)$

Setting the Weights

The SMART notation

Term frequency		Document frequency		Normalization	
n (natural)	$tf(t, d)$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(tf(t, d))$	t (idf)	$\log \frac{N}{df(t)}$	c (cosine)	$\frac{1}{\sqrt{w_1^2 + w_2^2 + \dots + w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times tf(t, d)}{\max_t(tf(t, d))}$	p (prob idf)	$\max\{0, \log \frac{N - df(t)}{df(t)}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases} 1 & \text{if } tf(t, d) > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^\alpha, \alpha < 1$
L (log avg)	$\frac{1 + \log(tf(t, d))}{1 + \log(\text{avg}_{t \in d}(tf(t, d)))}$				

- SMART notation of setting term weights: $d_{tf}d_{df}d_{norm} \cdot q_{tf}q_{df}q_{norm}$.
- A very standard weighting scheme: $lnc.ltc$ (used earlier as a sota scheme).

Setting the Weights

The SMART notation

Term frequency		Document frequency		Normalization	
n (natural)	$tf(t, d)$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(tf(t, d))$	t (idf)	$\log \frac{N}{df(t)}$	c (cosine)	$\frac{1}{\sqrt{w_1^2 + w_2^2 + \dots + w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times tf(t, d)}{\max_t(tf(t, d))}$			(pivoted cosine)	$1/u$
b (boolean)	$\begin{cases} 1 & \text{if } tf(t, d) > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^\alpha, \alpha < 1$
L (log avg)	$\frac{1 + \log(tf(t, d))}{1 + \log(\text{avg}_{t \in d}(tf(t, d)))}$				

- What if we use **lnc.lnc**?
- What if we use **lnc.ltc**?

- SMART notation of setting term weights: $d_{tf}d_{df}d_{norm} \cdot q_{tf}q_{df}q_{norm}$.
- A very standard weighting scheme: **lnc.ltc** (used earlier as a sota scheme).

Vector Space Model (VSM)

Examining the Independence Assumption

Core Assumption

- The Vector Space Model assumes that **terms are statistically independent**
- The occurrence of one term provides **no information** about the occurrence of another term
- Terms are treated as **orthogonal dimensions** in the vector space

Vector Space Model (VSM)

Examining the Independence Assumption

Core Assumption

- The Vector Space Model assumes that **terms are statistically independent**
- The occurrence of one term provides **no information** about the occurrence of another term
- Terms are treated as **orthogonal dimensions** in the vector space

Reality Check

Does this hold true for natural language?

Counterexamples

- Hong → Kong
- Microsoft → Windows
- hot → dog or weather
- artificial → intelligence

Mathematical Implication

- $P(t_1 \cap t_2) = P(t_1) \cdot P(t_2)$
- Covariance between terms = 0
- No semantic relationships captured
- Bag-of-words representation

Practical Impact

- **Simplifies computation**
- **Efficient for large collections**
- **But loses contextual meaning**

Vector Space Model (VSM)

Examining the Independence Assumption

Core Assumption

- The Vector Space Model assumes that **terms are statistically independent**
- The occurrence of one term provides **no information** about the occurrence of another term
- Terms are treated as **orthogonal dimensions** in the vector space

Reality Check

Does this

Modern Approaches

Word embeddings, BERT, and neural models **explicitly capture** term dependencies and semantic relationships.

Counterexamples

- Hong → Kong
- Microsoft → Windows
- hot → dog or weather
- artificial → intelligence

- No semantic relationships captured
- Bag-of-words representation

Practical Impact

- **Simplifies computation**
- **Efficient for large collections**
- **But loses contextual meaning**

Vector Space Model (VSM)

Summary and Key Principles

Core Concept

- **Any text** (document or query) can be represented as a **vector** in V-dimensional space
- Each dimension corresponds to a unique **term** in the vocabulary
- Documents become **points** in a high-dimensional vector space

Vector Space Model (VSM)

Summary and Key Principles

Core Concept

- **Any text** (document or query) can be represented as a **vector** in V-dimensional space
- Each dimension corresponds to a unique **term** in the vocabulary
- Documents become **points** in a high-dimensional vector space

Retrieval Mechanism

- **Rank documents** based on their **cosine similarity** to the query vector
- Higher cosine similarity = Better match = Higher ranking position
- Measures the **angle** between query and document vectors

Vector Space Model (VSM)

Summary and Key Principles

Core Concept

- **Any text** (document or query) can be represented as a **vector** in V-dimensional space
- Each dimension corresponds to a unique **term** in the vocabulary
- Documents become **points** in a high-dimensional vector space

Retrieval Mechanism

- **Rank documents** based on their **cosine similarity** to the query vector
- Higher cosine similarity = Better match = Higher ranking position
- Measures the **angle** between query and document vectors

Relevance Principle

- If a document is **geometrically similar** to the query, it is likely to be relevant
- Captures **topical relevance** through term overlap
- Assumption: Similar vectors represent similar topics

Vector Space Model (VSM)

Summary and Key Principles

Core Concept

- **Any text** (document or query) can be represented as a **vector** in V-dimensional space
- Each dimension corresponds to a unique **term** in the vocabulary
- Documents

Advantages

- Simple and intuitive
- Handles partial matches
- Robust to document length
- Efficient computation

Limitations

- Term independence assumption
- No semantic understanding
- Bag-of-words limitation
- Vocabulary sparsity issues

Retrieval Measure

- **Rank documents**
- Higher cosine similarity
- Measures the

Relevance Principle

- If a document is **geometrically similar** to the query, it is likely to be relevant
- Captures **topical relevance** through term overlap
- Assumption: Similar vectors represent similar topics